Secure File Transfer Monitoring System

Project Documentation and Audit Analysis

Domain: Blue Team / Data Loss Prevention

Technologies: Python, watchdog, hashlib, psutil

Name: _____ Roll No: _____

Date: October 2026

1. Abstract

File transfers inside and outside an organisation are a primary channel for data leakage, malware distribution and insider misuse. This project implements a Secure File Transfer Monitoring System (SFTMS) in Python. It watches the file system in real time, classifies each file as sensitive or normal, computes SHA-256 hashes before and after every transfer, checks each movement against an authorisation policy, and records every event in an audit trail. Policy violations raise graded alerts, and a final audit report summarises all activity.

In the controlled test described in Section 8, the toolkit processed 210 file events and raised 8 alerts (2 critical, 5 high, 1 warning). All 8 transfer-time integrity checks were recorded, and 2 hash mismatches were correctly flagged. A separate offline verification run also caught a file that was silently altered while the monitor was stopped.

2. Project Overview

Organisations face constant threats involving unauthorised uploads and downloads, data theft and file tampering. Typical examples are employees copying sensitive data outside the organisation, malware modifying or replacing files, and unauthorised transfers through USB devices, network shares or cloud-sync tools. A monitoring system helps detect suspicious transfers, unauthorised data movement, integrity violations and potential exfiltration attempts.

2.1 Objectives

- Log all file transfers performed on the system.
- Detect unauthorised movement of sensitive or restricted files.
- Implement file integrity checks using hashing (SHA-256, with MD5 selectable).
- Generate alerts on policy violations.
- Produce detailed audit logs and security reports.

2.2 Scope

Area	What the system does
File transfer logging	Records copy, move, delete, create and modify events with timestamp, source path, destination path, user and process name.
Unauthorised movement detection	Maintains lists of sensitive directories, extensions and keywords. Alerts when such files go to USB drives, network shares, cloud folders or unapproved locations, or are handled by unauthorised users.
Integrity checks	Calculates pre- and post-transfer hashes, detects tampering or corruption, and highlights mismatches.
Reporting and alerts	Writes JSONL and CSV logs, a text alert log, and a Markdown/HTML audit report with JSON summary.

3. Tools and Technologies

Component	Tool	Purpose
Language	Python 3.9+	Implementation of all modules

Component	Tool	Purpose
File system events	watchdog	Cross-platform observer for create, modify, delete and move events
Integrity	hashlib	SHA-256 (default) or MD5 file hashing in 1 MB chunks
Process / USB tracking	psutil (optional)	Best-effort process attribution and removable-drive detection
Windows extras	win32api / PowerShell (optional)	Owner lookup and directory enumeration on Windows
Documentation	Word, Draw.io, PowerPoint	Report, diagrams and final presentation

4. System Architecture

SFTMS is organised as a pipeline. The Event Monitor receives raw file system events from watchdog and passes them to the Detection Engine. The engine consults three helper modules (Classifier, Integrity Module and Policy configuration) and then sends its verdict to the Audit Logger, the Alert System and the Report Generator.

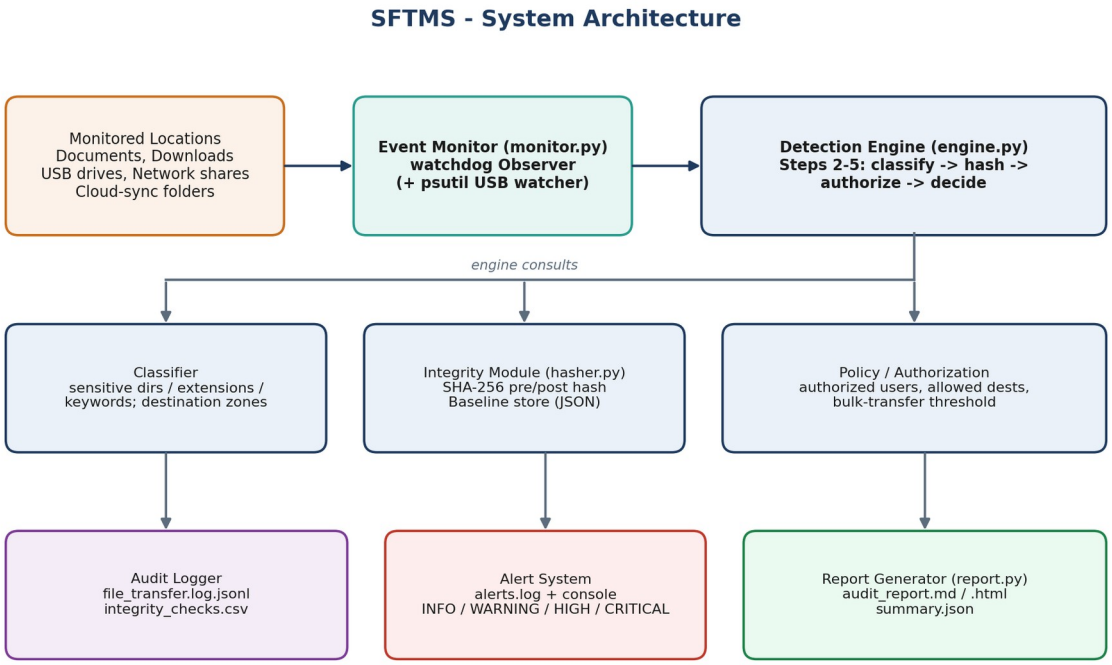


Figure 1: SFTMS system architecture

4.1 Source code modules

File	Responsibility
main.py	Command-line interface: baseline, monitor, verify, report
demo.py	Self-contained test harness that runs the ten scenarios in Section 8
config.json	Watched folders, sensitive rules, authorised users, USB/cloud/network prefixes, thresholds
sftms/monitor.py	watchdog observer and event handler, plus optional USB-insertion watcher

File	Responsibility
sftms/classifier.py	Sensitive-file classification and destination-zone classification
sftms/hasher.py	File hashing and the baseline store (trusted hashes, copy-detection index)
sftms/engine.py	Workflow Steps 2-5: classify, hash, authorise, log/alert; bulk-transfer detection
sftms/logger.py	JSONL audit log, text alert log, integrity CSV, colour console output
sftms/report.py	Final audit report generation and offline integrity verification

5. Workflow

Step	Name	Implementation
1	Monitor file system	watchdog Observer watches the configured folders recursively and reports create, modify, delete and move events.
2	Classify event	A file is sensitive if it lies in a sensitive directory, has a sensitive extension, or contains a sensitive keyword in its name. The destination is classified as removable, network, cloud, allowed or internal.
3	Integrity hashing	A SHA-256 hash is computed after the transfer and compared with the pre-transfer (baseline) hash.
4	Authorisation check	Compares user, destination zone and file sensitivity against the policy in config.json.
5	Logging and alerting	Every event is logged; violations also raise an alert with severity INFO, WARNING, HIGH or CRITICAL.
6	Final reporting	report.py aggregates the logs into audit_report.md/.html and summary.json.

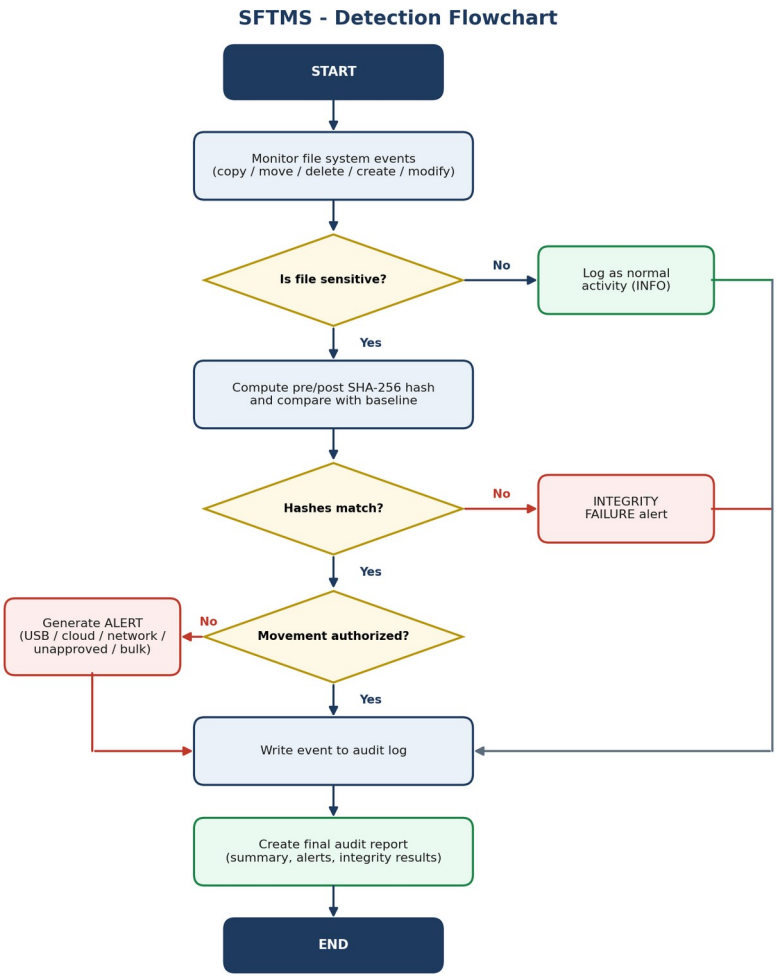


Figure 2: Detection flowchart

5.1 How copies are detected

File systems do not emit a copy event; a copy appears as a new file being created. SFTMS therefore keeps an index of the hash of every tracked file. When a new file appears and its hash matches an existing tracked file, the event is classified as a COPY and the original path becomes the source. If a new file has the same name as a tracked file but different content, it is treated as an altered or corrupted copy and produces an integrity failure.

6. Detection Rules and Alert Levels

Alert type	Severity	Trigger
SENSITIVE_TO_USB	CRITICAL	Sensitive file copied or moved to a removable drive
SENSITIVE_TO_CLOUD	CRITICAL	Sensitive file placed in a cloud-sync folder (Dropbox, OneDrive and similar)
SENSITIVE_TO_NETWORK	HIGH	Sensitive file sent to a network share
UNAUTHORIZED_MOVEMENT	HIGH	Sensitive file moved or copied outside the approved destinations
UNAUTHORIZED_USER	HIGH	User not in authorized_users handles a sensitive file
INTEGRITY_FAILURE	HIGH / CRITICAL	Post-transfer or post-modification hash differs from the trusted hash

Alert type	Severity	Trigger
SUSPICIOUS_BULK_TRANSFER	HIGH	Number of files reaching one unapproved directory within the time window reaches the threshold
SENSITIVE_DELETED	WARNING / HIGH	Sensitive file deleted (HIGH when by an unauthorised user)
SUSPICIOUS_OUTBOUND	WARNING	Non-sensitive file sent to USB, cloud or network location

7. Implementation Notes

Baseline. Before monitoring starts, python main.py baseline hashes every watched file and stores the result in logs/baseline.json. This is the trusted reference used for tamper detection and for the later verify command.

Stable-file wait. Large copies fire the create event before writing finishes. The engine waits until the file size stops changing before hashing, which avoids false mismatches.

Bulk detection. A sliding window per destination directory counts arrivals. When the count reaches bulk_threshold within bulk_window_seconds, one alert is raised for that window.

User and process. The user is taken from the operating system and the process name is resolved on a best-effort basis through psutil. In the demonstration, the scenario script labels actors (alice, intern01, bob and so on) through engine.actor_hints so that the log shows realistic attribution.

Running the toolkit.

```
pip install -r requirements.txt
python demo.py                # full demonstration with sample evidence
python main.py baseline       # hash all watched files
python main.py monitor        # live monitoring (Ctrl+C writes the report)
python main.py verify         # re-hash and detect tampering
```

8. Testing and Results

The demonstration script builds a sandbox folder tree, creates a baseline of seed files, starts the real watchdog observer, and performs the following scenarios through ordinary file operations. Alerts were produced by the system, not typed in.

#	Scenario	Actor	Outcome recorded by SFTMS
1	Copy a project note to Public	alice	COPY logged, INFO, hashes match
2	Archive the customer database to the approved Archive folder	alice	COPY logged, INFO (authorised), hashes match
3	Copy salary file to USB drive	intern01	CRITICAL: SENSITIVE_TO_USB
4	Sync private key into Dropbox folder	bob	CRITICAL: SENSITIVE_TO_CLOUD
5	Move customer database to Staging folder	admin	HIGH: UNAUTHORIZED_MOVEMENT (hash matched)
6	Malware appends bytes to private key	SYSTEM	HIGH: INTEGRITY_FAILURE (hash mismatch)
7	Document altered during copy to Public	alice	HIGH: INTEGRITY_FAILURE after copy
8	Non-sensitive draft copied to USB	alice	WARNING: SUSPICIOUS_OUTBOUND

#	Scenario	Actor	Outcome recorded by SFTMS
9	200 files dumped into an unknown folder	bob	HIGH: SUSPICIOUS_BULK_TRANSFER at the 50-file threshold; all 200 files logged
10	Delete archived sensitive database	intern01	HIGH: SENSITIVE_DELETED
11	Salary file edited while monitor was stopped	offline	python main.py verify reports MISMATCH

8.1 Summary statistics

Metric	Value
Total events logged	210
Events involving sensitive files	6
Alerts raised	8 (CRITICAL 2, HIGH 5, WARNING 1)
Transfer-time integrity checks	8 (6 match, 2 mismatch)
Offline verification	211 files re-hashed, 1 mismatch (scenario 11)

8.2 Evidence

The figures below are renderings of the toolkit's real console output and log files from the demonstration run (terminal-style captures). When you present on your own machine, replace them with live screenshots using the same commands.

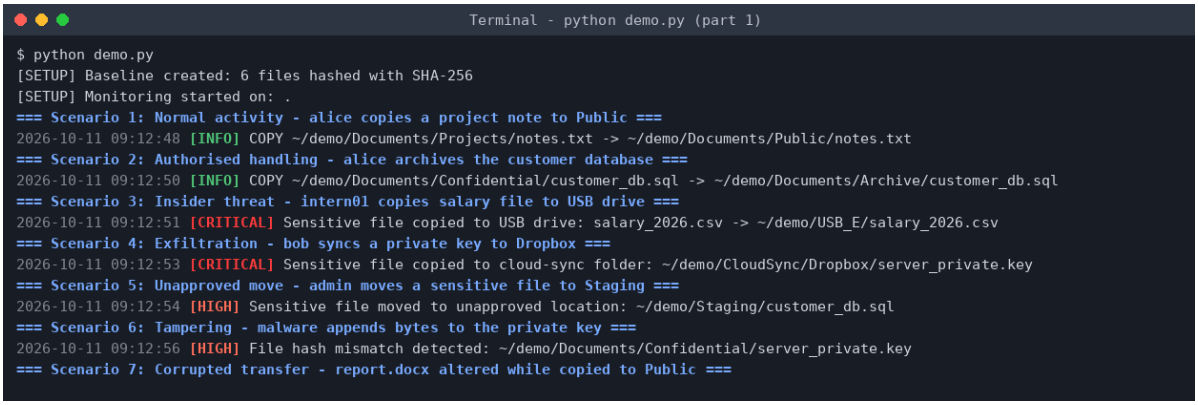


Figure 3: Live monitoring output, scenarios 1-3 and setup

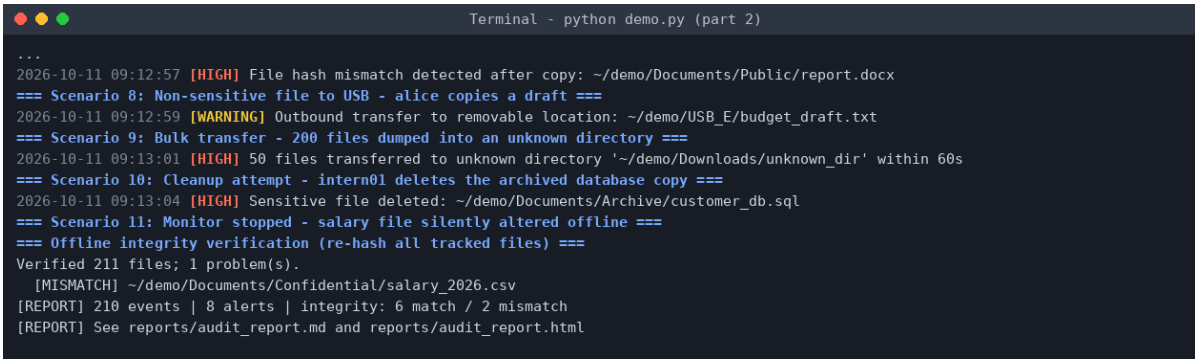


Figure 4: Live monitoring output, scenarios 4-11 and verification

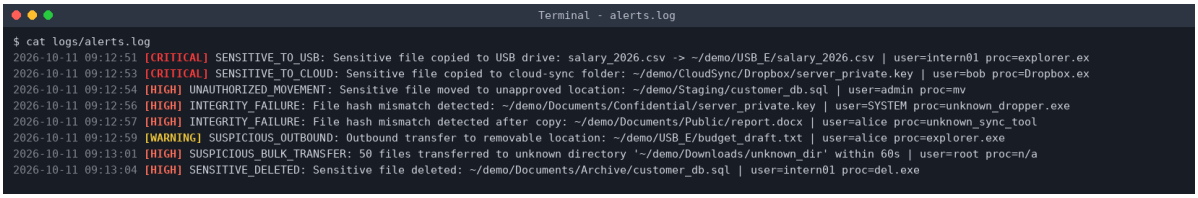


Figure 5: alerts.log

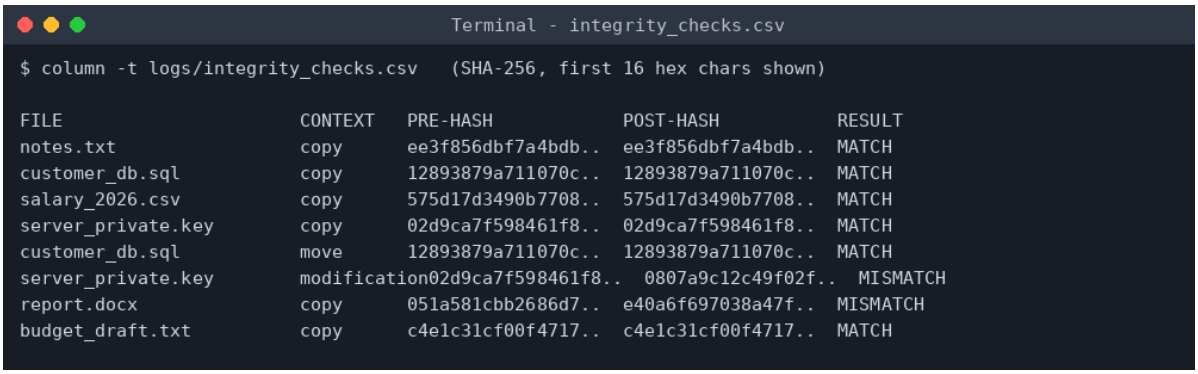


Figure 6: Integrity check evidence (pre and post hashes)

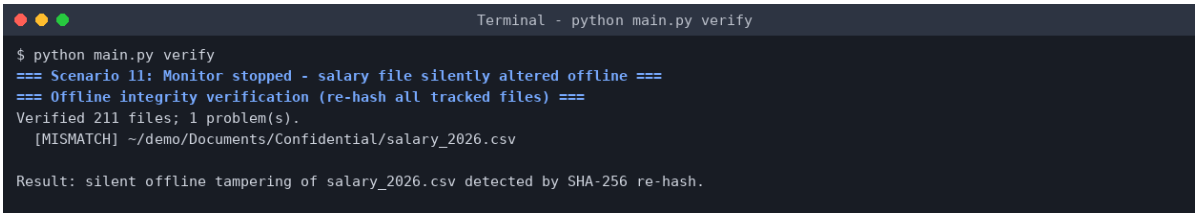


Figure 7: Offline verification detects silent tampering

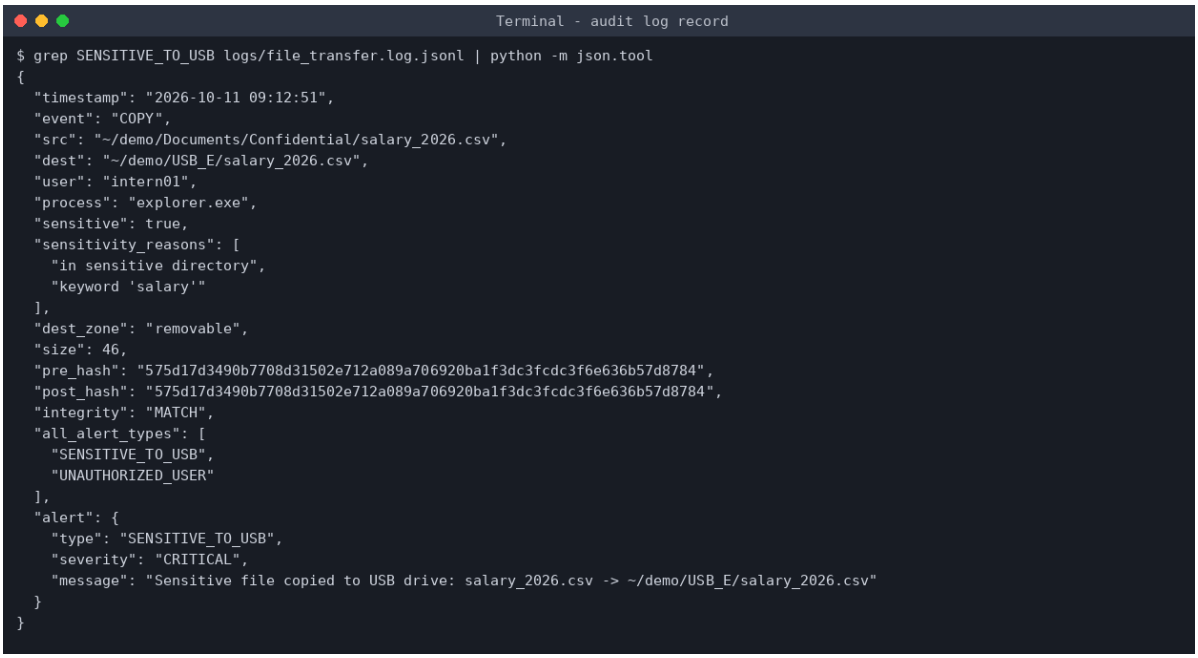


Figure 8: Structured audit log record for the USB alert

The complete Markdown and HTML audit report is in reports/audit_report.md and reports/audit_report.html.

9. Security Techniques and Learning Outcomes

- File system activity monitoring and sensitive data movement tracking.
- Tamper detection through hashing, including pre- and post-transfer comparison.
- Unauthorised access alerting and insider-threat detection.
- Data loss prevention (DLP) concepts: classification, policy, enforcement points and audit.
- Real-world file auditing: structured logs, severity grading and reporting.

10. Limitations and Future Work

- watchdog does not report reads. Detecting plain read access to sensitive files would need OS auditing (Linux auditd, Windows Event ID 4663) fed into the same engine.
- A move between two separately watched roots can arrive as a create plus a delete. Watching a common parent folder, or pairing such events by hash, avoids this.
- Process attribution through psutil is best effort, because short-lived processes may exit before the event is handled.
- Policy is path and name based. Content inspection (for example pattern matching for card numbers) would improve classification.
- Possible extensions: e-mail or webhook alerts, a web dashboard, and blocking rather than only detecting.

11. Conclusion

SFTMS meets the project objectives: it logs file transfers, detects unauthorised movement of sensitive files, verifies integrity with hashing, raises graded alerts and produces audit reports. The controlled test shows that it catches the main exfiltration paths (USB, cloud, unapproved folders, bulk dumps) and both runtime and offline tampering. It is a practical foundation for a host-based DLP monitoring capability.

12. References

- watchdog documentation: https://python-watchdog.readthedocs.io
- Python hashlib documentation: https://docs.python.org/3/library/hashlib.html
- psutil documentation: https://psutil.readthedocs.io
- NIST FIPS 180-4, Secure Hash Standard (SHA-256)